



Universidade do Minho

Mestrado em Engenharia Eletrónica Industrial e Computadores

MEMGUARD

Memory Security in RISC-V Systems

Projeto Integrador em Engenharia Eletrónica Industrial
e Computadores

Autores:

João Mendes
PG60206

Tiago Oliveira
PG60217

Orientadores:

Tiago Gomes

Luís Cunha
Manuel Rodriguez

Braga, junho de 2026

Resumo

Em sistemas embebidos baseados em microcontroladores, a arquitetura de hardware carece tipicamente de suporte para memória virtual de modo a cumprir os requisitos estritos de determinismo, simplicidade e previsibilidade exigidos em tempo real. Esta limitação estrutural impede a aplicação dos mecanismos clássicos de isolamento entre *tasks*, expondo os sistemas a riscos de *safety* e *security* quando múltiplos componentes da pilha de *software* partilham a memória como recurso físico. Este trabalho explora experimentalmente os mecanismos de proteção de memória da arquitetura RISC-V, com foco no *Physical Memory Protection* (PMP), num ambiente baseado no processador CVA6. Foram implementados e comparados dois sistemas operativos de tempo real (RTOS): o FreeRTOS, que não dispõe de isolamento de memória nativo, e o Zephyr, com suporte nativo para isolamento de memória através do PMP. A comparação abrange cenários de comportamento adversarial entre *tasks*. O sistema foi validado num ambiente de simulação baseado em Verilator e em *hardware* real numa FPGA Genesys2. No decurso desta validação, o FreeRTOS foi estendido com a execução de *tasks* em U-Mode e foi identificada e corrigida uma anomalia de temporização no *RTL* do CVA6, que classificava incorretamente escritas ilegais como leituras, comprometendo a integridade do mecanismo PMP. Os resultados demonstram que a configuração correta das entradas PMP garante isolamento efetivo entre os componentes da pilha de *software*, bloqueando acessos indevidos e permitindo a deteção imediata de violações, ao passo que a ausência de configuração permite a exploração das vulnerabilidades sem qualquer deteção.

Conteúdo

1	Introdução	4
1.1	Contextualização e Motivação	4
1.2	Objetivos	4
1.3	Organização do Relatório	5
2	Estado da Arte	6
2.1	Arquitetura RISC-V	6
2.1.1	Níveis de Privilegio	6
2.1.2	Physical Memory Protection	6
2.2	RTOSes em Sistemas Embebidos	7
2.3	Trabalhos Relacionados	8
2.4	O Processador CVA6	8
2.5	Ferramentas e Tecnologias de Suporte	9
3	Análise do Problema	10
3.1	Definição do Problema	10
3.2	Requisitos e Restrições	10
3.3	Cenários de Ataque	10
4	Conceção da Solução	12
4.1	Arquitetura Geral	12
4.2	FreeRTOS com Extensão para U-Mode	12
4.3	Zephyr com Suporte Nativo a PMP	12
4.4	Design das <i>tasks</i>	12
5	Descrição da Solução	14
5.1	Ambiente de Desenvolvimento e Fluxo de Trabalho	14
5.2	Correção ao RTL do CVA6	14
5.2.1	Diagnóstico	14
5.2.2	Correção	16
5.3	Implementação dos Cenários de Demonstração	16
5.3.1	Configuração de Isolamento no Zephyr	17
5.3.2	Escrita na <i>Stack</i> de Outra <i>Task</i>	17
5.3.3	Escrita em Variável Global do Kernel	18
5.3.4	Corrupção do TCB	18
5.3.5	Bug Acidental	18
5.4	Extensão do FreeRTOS para U-Mode	18
6	Testes e Discussão	21
6.1	Metodologia de Testes	21
6.2	Simulação: Escrita em Variável Global do Kernel	21
6.3	Hardware: Bug Acidental	21
6.4	Hardware: Extensão do FreeRTOS para U-Mode	22
6.5	Demonstração Física: Escrita na <i>Stack</i> de Outra <i>Task</i>	23
6.6	Demonstração Física: Corrupção do TCB	24
6.7	Validação dos Objetivos	25
7	Conclusão	26
7.1	Trabalho Futuro	26

Lista de Figuras

1.1	Domínios <i>safety-critical</i> onde o isolamento de memória é crítico.	4
2.1	Níveis de privilégio RISC-V e transições entre modos.	6
2.2	Estrutura do registo <code>pmpcfg</code>	7
3.1	Cenário sem PMP: acesso indevido não é bloqueado.	10
3.2	Cenário com PMP: violação detetada e contida pelo hardware.	11
4.1	<i>Context switch</i> : FreeRTOS vs Zephyr ao nível do PMP.	12
5.1	Formas de onda antes da correção	15
5.2	Formas de onda após a correção	16
6.1	Simulação Verilator: escrita em variável global do kernel.	21
6.2	FreeRTOS: <i>overflow</i> silencioso; dados de outra <i>task</i> corrompidos sem deteção.	22
6.3	Zephyr (hardware): deteção pelo PMP e diagnóstico simbólico a partir do <code>mepc</code>	22
6.4	Extensão do FreeRTOS para U-Mode: M-Mode vs bloqueio em U-Mode ao executar instruções privilegiadas.	22
6.5	Montagem física: FPGA Genesys2 a gerar PWM via GPIO, ponte H L298N e motor DC. . .	23
6.6	Demonstração física (motor): escrita na <i>stack</i> de outra <i>task</i>	24
6.7	Demonstração física (motor): corrupção do TCB no FreeRTOS; <i>scheduler</i> com <i>stack pointer</i> inválido cai no <i>trap handler</i>	24

Lista de Tabelas

2.1	CSRs relevantes para este trabalho.	7
2.2	Comparação entre FreeRTOS e Zephyr.	8
3.1	Requisitos funcionais e não-funcionais do sistema.	10
5.1	Sinais relevantes no diagnóstico da dessincronização PMP	15
5.2	Cenários de demonstração e comportamento esperado por RTOS.	17
6.1	Validação dos objetivos do trabalho.	25

1. Introdução

1.1 Contextualização e Motivação

Os microcontroladores carecem tipicamente de suporte de hardware para memória virtual devido a requisitos de determinismo, simplicidade e previsibilidade exigidos pelos sistemas de tempo real. Esta característica contrasta com os processadores de uso geral, onde a unidade de gestão de memória (MMU) garante isolamento através da tradução de endereços. Num microcontrolador (MCU), na ausência de memória virtual, as *tasks* podem aceder livremente a qualquer endereço físico, incluindo regiões de memória que não lhes pertencem. O problema não reside na partilha de espaços de endereçamento, já que cada *task* ocupa a sua própria região, mas sim na visibilidade irrestrita que uma componente da pilha de software tem sobre o espaço de endereçamento de outra. Sem barreiras de proteção ativas, qualquer *task* pode ler ou modificar arbitrariamente a memória das restantes ou do próprio sistema operativo. Mesmo quando o hardware disponibiliza mecanismos de isolamento, como o PMP na arquitetura RISC-V, o software nem sempre faz um uso eficiente deles.

Esta ausência de isolamento representa um risco crescente em domínios *safety-critical*, ilustrados na Figura 1.1. Em 2024, falhas de *software* foram responsáveis por 44% de todos os veículos recolhidos do mercado nos Estados Unidos, num total de 15 milhões de unidades, representando um aumento de 80% face ao ano anterior [1]. Em contextos médicos e industriais, onde falhas em sistemas embebidos críticos também têm motivado desvios severos de segurança operacional [2], as consequências de uma *task* comprometida podem ser igualmente graves, seja por corrupção accidental ou maliciosa de dados críticos (*safety*), seja por acessos indevidos a informação sensível (*security*).

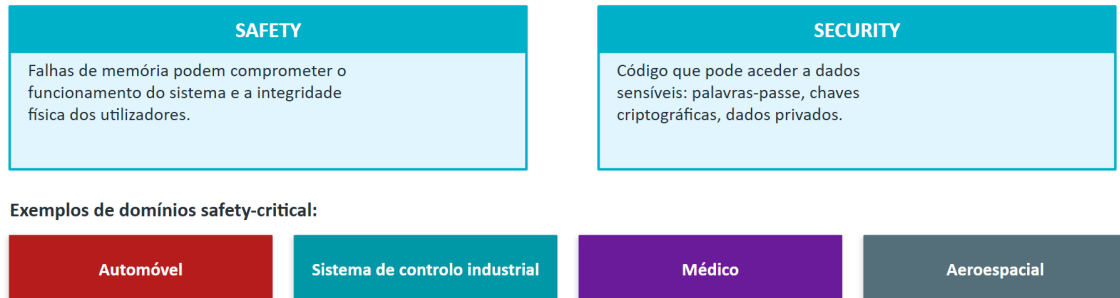


Figura 1.1: Domínios *safety-critical* onde o isolamento de memória é crítico.

A arquitetura RISC-V disponibiliza dois mecanismos complementares para mitigar este problema: os níveis de privilégio, que garantem o isolamento ao nível dos registos do CPU e da execução de instruções, e a primitiva PMP, que permite a uma componente privilegiada de *software* restringir os acessos de cada *task* a uma região física delimitada [3]. A eficácia desta proteção depende, de forma determinante, de como cada RTOS utiliza estes mecanismos, aspeto que constitui o eixo central deste trabalho sobre o processador CVA6 [4].

1.2 Objetivos

O presente trabalho tem como objetivos principais:

1. Analisar os níveis de privilégio da arquitetura RISC-V e o modelo de configuração da primitiva PMP no processador CVA6;
2. Configurar o FreeRTOS para o processador CVA6
3. Estender o FreeRTOS com o suporte à execução de *tasks* em *User Mode* (U-Mode);
4. Configurar o Zephyr para o CVA6, ativando o suporte nativo ao PMP com reconfiguração por

task durante o *context switch* para assegurar o isolamento de memória;

5. Desenvolver *tasks* de demonstração com comportamento adversarial, permitindo avaliar o impacto prático da presença de isolamento de memória face à sua ausência;
6. Comparar experimentalmente o FreeRTOS e o Zephyr no que concerne ao isolamento de memória, tanto em ambiente de simulação como em *hardware* real numa FPGA Genesys2.

Estes objetivos são validados individualmente no Capítulo 6.

1.3 Organização do Relatório

O Capítulo 2 apresenta os fundamentos da arquitetura RISC-V, o mecanismo PMP, os RTOSes utilizados, os trabalhos relacionados e as ferramentas. O Capítulo 3 define o problema, os requisitos e os cenários de ataque. O Capítulo 4 descreve a arquitetura da solução e as decisões de *design*. O Capítulo 5 detalha a implementação. O Capítulo 6 apresenta os testes e resultados. O Capítulo 7 conclui e identifica trabalho futuro.

2. Estado da Arte

Este capítulo constrói o enquadramento técnico necessário para compreender o trabalho desenvolvido. Parte da arquitetura RISC-V e dos seus mecanismos de privilégio e proteção, contextualiza o funcionamento de sistemas operativos de tempo real (RTOS), analisa os trabalhos relacionados, detalha o processador CVA6 e descreve as ferramentas de suporte utilizadas.

2.1 Arquitetura RISC-V

O RISC-V é uma arquitetura de conjunto de instruções (*Instruction Set Architecture*, ISA) aberta, modular e livre de *royalties* [5]. A sua especificação pública permite que qualquer entidade implemente *hardware* e/ou *software* sem restrições de licenciamento, o que tem impulsionado a sua adoção em contextos académicos e industriais. A arquitetura organiza-se em torno de uma base obrigatória, RV32I ou RV64I consoante a largura de palavra, complementada por extensões opcionais que permitem adaptar uma implementação a domínios com requisitos muito particulares.

Do ponto de vista de segurança da arquitetura, destacam-se dois mecanismos centrais para o isolamento de sistemas: os níveis de privilégio, que definem a quais registos e instruções o *software* tem acesso, e o PMP, que restringe os acessos à memória física.

2.1.1 Níveis de Privilégio

A arquitetura base define três níveis de privilégio: M-Mode (*Machine*), S-Mode (*Supervisor*) e U-Mode (*User*) [3]. Com a inclusão da *Hypervisor Extension*, o modelo é estendido para suportar a execução de hipervisores e máquinas virtuais. Em MCUs, apenas M-Mode e U-Mode estão tipicamente disponíveis. O M-Mode tem acesso irrestrito a todos os recursos de *hardware* e é onde o *kernel* do RTOS executa. O U-Mode destina-se às *tasks* das aplicações, com acesso restrito e sem capacidade de executar instruções privilegiadas. As transições entre modos são controladas: de U-Mode para M-Mode via exceção ou instrução `ecall`, e o retorno via `mret`, que restaura o contexto e o nível de privilégio anterior (Figura 2.1).

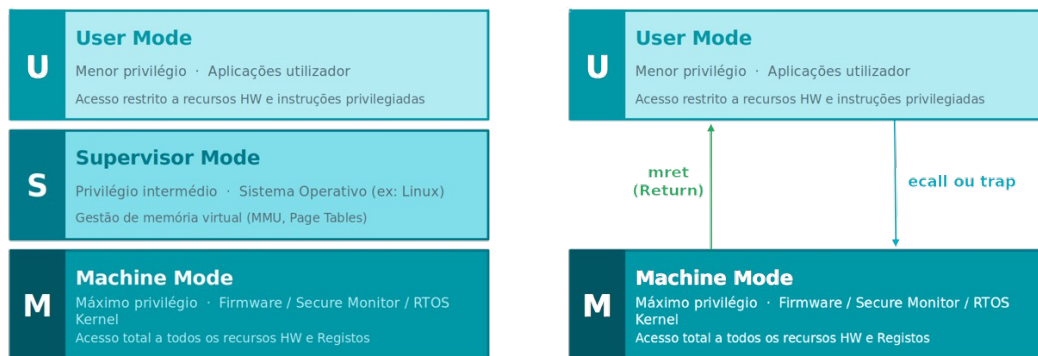


Figura 2.1: Níveis de privilégio RISC-V e transições entre modos.

Para suportar as operações de configuração do sistema e o tratamento de exceções, a arquitetura disponibiliza um conjunto de registos de controlo e estado (CSRs), descritos na Tabela 2.1.

A separação de privilégios é condição necessária para o isolamento de memória, mas não suficiente: por si só não restringe a que memória cada modo acede. Esse controlo cabe ao PMP, descrito na secção seguinte.

2.1.2 Physical Memory Protection

Em ambientes baseados em MCUs, a ausência de uma MMU implica que toda a memória física é partilhada por todas as *tasks* do sistema. Neste contexto, o PMP é o mecanismo nativo da

Tabela 2.1: CSRs relevantes para este trabalho.

Registo	Função	Utilização neste trabalho
mstatus	Estado global; campo MPP define o privilégio ao executar <code>mret</code>	Configurar U-Mode no lançamento de <i>tasks</i>
mepc	Endereço da instrução que causou a exceção ou <i>entry point</i> de uma <i>task</i>	Definir o ponto de entrada de cada <i>task</i>
mcause	Causa da exceção: 0x5 = <i>Load Access Fault</i> ; 0x7 = <i>Store/AMO Access Fault</i>	Identificar o tipo de violação PMP
mtval	Em violações PMP, contém o endereço acedido ilegalmente	Determinar o endereço alvo do acesso indevido
mtvec	Endereço do <i>trap handler</i>	Encaminhar exceções PMP para tratamento

arquitetura RISC-V para controlo de acessos à memória física [3], sendo a única ferramenta de *hardware* capaz de impor isolamento explícito entre componentes.

Configurado em M-Mode, o PMP define regiões com permissões independentes para modos menos privilegiados. A especificação define que o M-Mode não é restringido pelas entradas PMP; contudo, quando o bit L está ativo, as permissões aplicam-se também ao M-Mode e a configuração torna-se imutável até ao próximo *reset*. Isto permite ao M-Mode proteger-se de si próprio, por exemplo, caso o *kernel* do RTOS seja comprometido.

Cada entrada PMP é definida por dois CSRs: `pmpaddr`, que especifica o endereço ou os limites da região, e `pmpcfg`, que codifica as permissões e o modo de endereçamento em 8 bits por entrada (Figura 2.2). A especificação suporta até 64 entradas.

Figura 2.2: Estrutura do registo `pmpcfg`.

Os campos R, W e X definem as permissões de leitura, escrita e execução. O campo A determina o modo de endereçamento: TOR (*Top of Range*) ou NAPOT (*Naturally Aligned Power-Of-Two*). Quando uma *task* em U-Mode acede a um endereço sem permissão, o PMP interceta o pedido antes de chegar ao barramento e gera um *access fault*. Este evento desencadeia uma transição forçada de U-Mode para M-Mode, encaminhando a exceção para o *trap handler* via `mtvec`. Esta transição é determinante, dado que é o que permite ao *kernel* assumir o controlo do tratamento da violação. O *handler* identifica o tipo de violação através de `mcause` e o endereço do acesso ilegal através de `mtval` (ver Tabela 2.1), permitindo parar o sistema de forma controlada.

2.2 RTOSes em Sistemas Embebidos

Um RTOS garante respostas determinísticas dentro de prazos definidos, privilegiando a previsibilidade temporal. Cada *task* tem prioridade, pilha e contexto próprios; o *scheduler* decide qual executa a cada instante, e a rotina de *context switch* guarda o contexto da *task* atual e restaura o da seguinte. É precisamente neste momento que o suporte ao PMP faz diferença: um RTOS que reconfigure as

entradas PMP por *task* garante isolamento dinâmico; um que não o faça deixa a memória exposta.

O FreeRTOS [6] é um dos RTOSes mais adotados em sistemas embebidos, com uma arquitetura minimalista em C puro e suporte a centenas de plataformas [7]. No contexto RISC-V, executa o *kernel* e todas as *tasks* em M-Mode, sem fazer uso dos níveis de privilégio distintos nem configurar o PMP no *context switch*.

O Zephyr [8], desenvolvido sob a Linux Foundation, dispõe de suporte nativo ao PMP em RISC-V [9]. As entradas PMP são configuradas por *task*, garantindo que cada uma acede apenas à sua própria região. Quando uma violação ocorre, é acionado o mecanismo de detecção descrito na Secção 2.1.2 e o sistema é interrompido. A Tabela 2.2 resume as diferenças relevantes entre os dois RTOSes.

Tabela 2.2: Comparação entre FreeRTOS e Zephyr.

Critério	FreeRTOS	Zephyr
Modo do kernel	M-Mode	M-Mode
Modo das <i>tasks</i>	M-Mode	U-Mode
Suporte a PMP	Não nativo	Nativo
Isolamento	Não	Por região física
<i>Context switch</i> PMP	Sem reconfiguração	Reconfigura por <i>task</i>
Resposta a violação	Silenciosa	Access fault → halt do sistema

2.3 Trabalhos Relacionados

Em 2019, Zhou et al. [10] analisaram a utilização das Unidades de Proteção de Memória (MPU) na arquitetura ARM em sistemas embebidos. Os autores concluíram que a MPU é raramente adotada em produtos reais, atribuindo-o sobretudo à granularidade grosseira da proteção que oferece e ao facto de o acesso às interfaces de programação de aplicações (*Application Programming Interfaces* APIs) do *kernel* exigir uma elevação do nível de privilégio da *task*. Estas limitações são relevantes para o presente trabalho, uma vez que se manifestam de forma análoga na extensão para U-Mode aqui desenvolvida.

Em 2024, Larmann [11] integrou o mecanismo de PMP no FreeRTOS, com reconfiguração dinâmica das entradas por *task* sobre o núcleo CV32E40S. O autor validou o isolamento resultante recorrendo a *tasks* maliciosas simuladas. O presente trabalho complementa esta abordagem ao comparar diretamente o FreeRTOS e o Zephyr sobre o mesmo *hardware*, em simulação e em FPGA, e ao corrigir uma anomalia no *RTL* do CVA6.

Em 2022, Cheang et al. [12] aplicaram técnicas de verificação formal ao módulo PMP do Rocket Chip. Os autores demonstraram que mesmo implementações de referência podem apresentar desvios subtis face à especificação RISC-V. Este resultado reforça a relevância da anomalia identificada no presente trabalho, em que escritas ilegais eram classificadas como leituras e executadas sem qualquer exceção.

2.4 O Processador CVA6

O CVA6 [4, 13] é um *core* RISC-V de classe de aplicação, desenvolvido pelo OpenHW Group, que implementa as versões RV64 e RV32 da ISA. O *core* na sua versão base suporta os três modos de privilégio (M/S/U) e dispõe de uma MMU, o que o torna capaz de executar sistemas operativos completos como o Linux. A nível microarquitetural, organiza-se num *pipeline* de seis estágios, com emissão única (*single issue*) e execução *in-order*. As unidades funcionais podem, contudo, concluir o *write-back* fora de ordem, sendo a ordem arquitetural reposta por uma estrutura de *scoreboard* antes do *commit*.

A variante utilizada neste trabalho é o CVA6 (32 bits), cujos parâmetros são definidos em tempo de

síntese (detalhes no Capítulo 5), nomeadamente os níveis de privilégio disponíveis, a disponibilidade de S-Mode via parâmetro **RVS** e o número de entradas PMP. A configurabilidade do *core* em tempo de síntese é uma das suas características centrais, permitindo adaptá-lo a requisitos específicos do sistema.

2.5 Ferramentas e Tecnologias de Suporte

O desenvolvimento e a validação do sistema recorreram às seguintes ferramentas:

- **Verilator** [14] : simulador *cycle-accurate* do *RTL* do CVA6, usado no desenvolvimento e validação funcional;
- **Vivado** : software empregue para os processos de síntese e implementação do *bitstream* na FPGA;
- **FPGA Genesys2** [15] (Xilinx Kintex-7) : plataforma de execução em *hardware* real para validação física.

3. Análise do Problema

3.1 Definição do Problema

Num MCU RISC-V em que as *tasks* executam em M-Mode, como acontece na configuração base do FreeRTOS, o PMP não é aplicado e cada *task* tem acesso irrestrito ao espaço de endereçamento físico, incluindo a memória do kernel e de outras *tasks*. Um *bug* ou código malicioso pode corromper dados críticos ou aceder a informação sensível sem qualquer deteção pelo hardware. O problema não é a ausência do mecanismo, dado que o PMP existe no CVA6, mas a sua não utilização por parte do RTOS.

A ameaça concreta abordada neste trabalho é a de uma *task* adversarial que, tirando partido desta ausência de configuração, tenta ler ou corromper a memória de outra *task* ou do próprio kernel, sem que o hardware o detete ou bloqueie.

3.2 Requisitos e Restrições

Tabela 3.1: Requisitos funcionais e não-funcionais do sistema.

Tipo	ID	Requisito
Funcional	RF1	Cada <i>task</i> em U-Mode acede apenas à sua região de memória.
Funcional	RF2	Acessos fora da região autorizada são detetados pelo hardware.
Funcional	RF3	Após violação, o sistema para de forma controlada e regista a causa.
Funcional	RF4	O kernel executa em M-Mode com acesso total à memória.
Funcional	RF5	As permissões PMP são reconfiguradas a cada <i>context switch</i> .
Não-func.	RNF1	A implementação deve utilizar o CVA6
Não-func.	RNF2	Validação com FreeRTOS e Zephyr.
Não-func.	RNF3	Validação em simulação (Verilator) e FPGA (Genesys2).
Não-func.	RNF4	Comportamento adversarial inclui leitura e escrita fora da região.

3.3 Cenários de Ataque

A análise considera dois cenários complementares que correspondem à ausência e à presença de configuração PMP. Em ambos, existe uma *task* legítima (Task A) e uma *task* adversarial (Task B) que tenta aceder à região de memória do Kernel.

No primeiro cenário (Figura 3.1), sem PMP configurado, o acesso da Task adversarial chega à memória sem qualquer bloqueio. A memória do Kernel é comprometida e o sistema não deteta a violação.

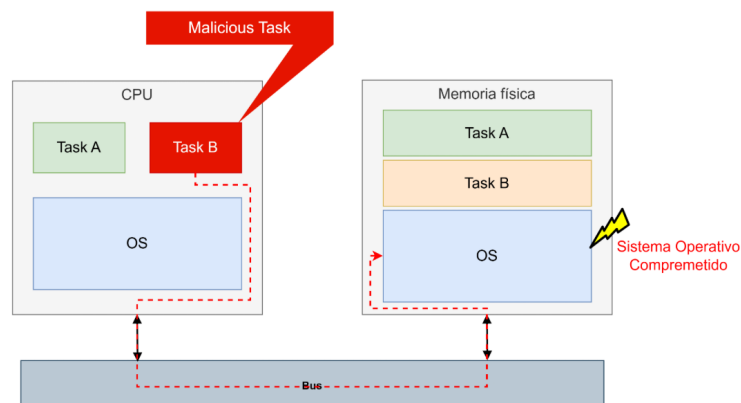


Figura 3.1: Cenário sem PMP: acesso indevido não é bloqueado.

No segundo cenário (Figura 3.2), com PMP ativo, o mesmo acesso é interceptado pela unidade PMP, acionando o mecanismo de detecção descrito na Secção 2.1.2: a violação é sinalizada e o sistema parado de forma controlada. A Task A não chega a ser perturbada.

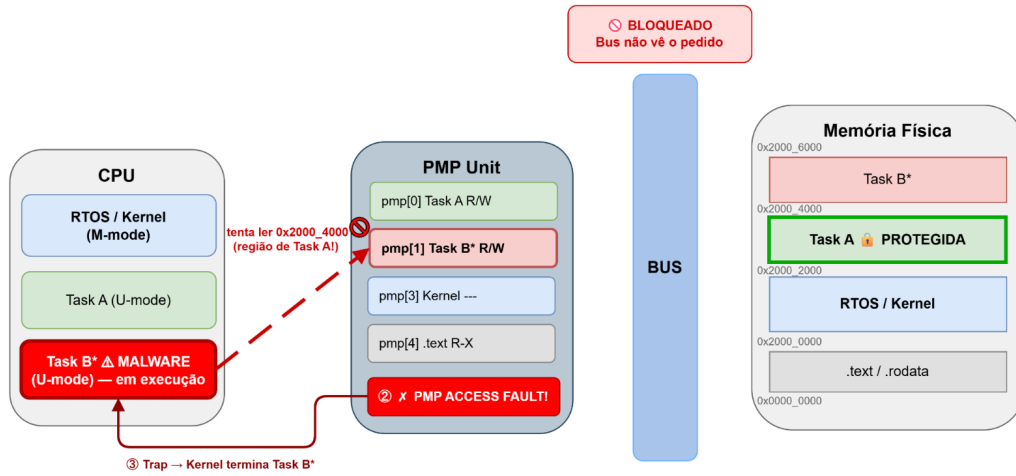


Figura 3.2: Cenário com PMP: violação detetada e contida pelo hardware.

Os cenários concretos que materializam esta ameaça são definidos na Secção 4.4 e implementados em detalhe na Secção 5.3.

4. Conceção da Solução

4.1 Arquitetura Geral

A solução assenta em dois cenários executados sobre o mesmo hardware, o CVA6, diferenciando exclusivamente na forma como o software privilegiado utiliza o PMP. No FreeRTOS, o kernel e as *tasks* executam em M-Mode, sem qualquer configuração PMP, este é o cenário de base utilizado nos testes de ataque. No Zephyr, o kernel executa em M-Mode e as *tasks* em U-Mode, com o PMP reconfigurado por *task* a cada *context switch*. A extensão do FreeRTOS para U-Mode, descrita na Secção 5.4, é uma contribuição adicional que demonstra que a separação de privilégio sem PMP é insuficiente para garantir isolamento, mas não substitui o cenário de base.

4.2 FreeRTOS com Extensão para U-Mode

O FreeRTOS executa, na sua configuração base, o kernel e todas as *tasks* em M-Mode. Como contribuição adicional ao trabalho de base, o FreeRTOS foi modificado neste projeto para lançar *tasks* em U-Mode, reduzindo o seu nível de privilégio no arranque. Esta extensão estabelece uma distinção base de privilégios, com o kernel em M-Mode e as *tasks* em U-Mode. Uma vez que nenhuma entrada PMP restritiva é configurada, o espaço de endereçamento permanece integralmente acessível a todas as *tasks*, servindo de referência para a comparação com o Zephyr.

4.3 Zephyr com Suporte Nativo a PMP

No Zephyr, a cada *context switch*, o kernel reconfigura as entradas PMP com os endereços e permissões da *task* seguinte. A decisão de design assenta na utilização do mecanismo nativo de isolamento do Zephyr para RISC-V, sem modificações ao *port* existente, de modo a que o comportamento observado seja representativo de uma implantação real. Quando uma *task* em U-Mode viola as permissões atribuídas, o PMP gera um *access fault* que culmina na invocação do tratamento de erro fatal nativo do Zephyr. A Figura 4.1 compara o *context switch* nos dois cenários, evidenciando onde reside a diferença operacional.

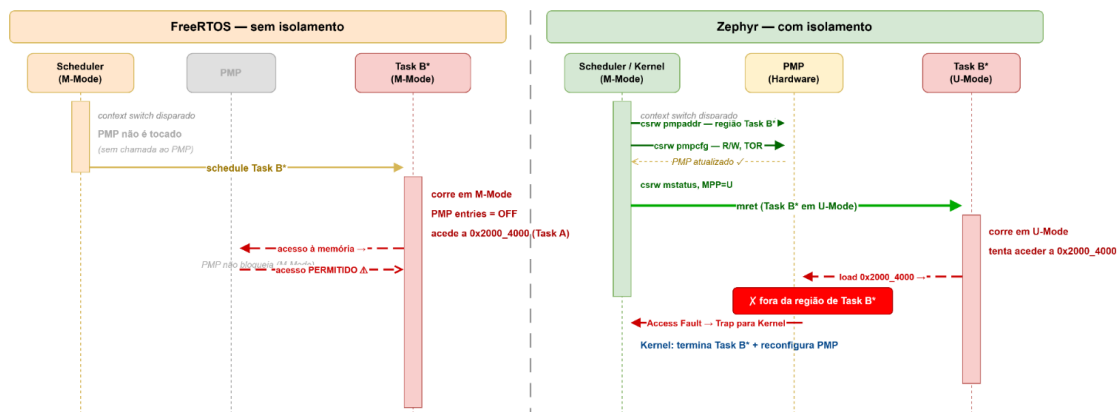


Figura 4.1: *Context switch*: FreeRTOS vs Zephyr ao nível do PMP.

4.4 Design das *tasks*

O sistema inclui duas *tasks* de demonstração. A Task A é a *task* legítima. Em cenários que exigem coordenação, expõe deliberadamente o endereço de uma variável local através de uma variável partilhada, simulando a condição de um registo de depuração acessível externamente. Esta simplificação isola o efeito do isolamento PMP do problema de descoberta de endereços, que está fora do âmbito deste trabalho. A Task Adv é a *task* adversarial. Os alvos cobrem as três regiões de memória sobre as quais uma *task* pode exercer impacto: a *stack* privada de outra *task*, o espaço do

kernel, e a memória local por erro de programação. Esta cobertura permite avaliar o mecanismo PMP tanto em contexto intencional como acidental. Todos os ataques assentam em escritas, por serem diretamente observáveis através da corrupção de dados; leituras ilegais são bloqueadas pelo mesmo mecanismo. No Zephyr, a Task Adv recebe permissões R/W apenas sobre a sua própria região e sobre a partição partilhada de coordenação; qualquer acesso fora desse espaço gera um Store Access Fault. No FreeRTOS, a entrada PMP cobre todo o espaço de endereçamento com permissões R/W/X e a Task Adv pode escrever em qualquer endereço sem restrição.

5. Descrição da Solução

5.1 Ambiente de Desenvolvimento e Fluxo de Trabalho

O desenvolvimento e a validação do sistema decorreram em dois ambientes complementares: simulação do CVA6 via Verilator e execução em hardware real na FPGA Genesys2. A variante do CVA6 utilizada consiste numa configuração de 32 bits que integra as extensões I (instruções base de inteiros), M (multiplicação e divisão inteira), A (instruções atômicas) e C (instruções comprimidas), dispondo de 16 entradas PMP, parâmetros fixados em tempo de síntese conforme referido na Secção 2.5.

Em simulação, o RTL do CVA6 é convertido pelo Verilator num modelo funcional *cycle-accurate* implementado em C++, o qual é posteriormente compilado num executável que simula o comportamento do processador. Os binários do FreeRTOS e do Zephyr são então executados diretamente sobre este modelo. Este ambiente permite iterações rápidas de desenvolvimento e depuração, uma vez que dispensa o processo de síntese e dá visibilidade total sobre os sinais internos do processador, propriedade que se revelou determinante na Secção 5.2.

Em hardware real, o *bitstream* do CVA6 é gerado e carregado na Genesys2 através do Vivado. O carregamento e a depuração do código que implementa toda a pilha de software são feitos via JTAG, com o OpenOCD [16] a servir de ponte entre a FPGA e o GDB: O GDB liga-se ao OpenOCD, carrega o ficheiro *Executable and Linkable Format* (ELF) para a memória e controla a execução, com acesso a ferramentas de depuração e aos símbolos do código C. O *output* do sistema é observado através da porta série (UART).

Este fluxo permitiu desenvolver e validar primeiro em simulação e replicar depois os mesmos testes em hardware, garantindo que os resultados do Capítulo 6 são diretamente comparáveis entre ambientes.

5.2 Correção ao RTL do CVA6

Durante a validação dos cenários com PMP ativo, foi identificado um comportamento anómalo no reporte de violações: *stores* ilegais geravam `mcause = 0x5` (*Load Access Fault*) em vez de `mcause = 0x7` (*Store Access Fault*), e o registo `mtval` continha um endereço incorreto, e não o endereço do acesso que falhou. Esta secção descreve o diagnóstico e a correção aplicada ao RTL do CVA6.

5.2.1 Diagnóstico

Foram primeiro excluídas as causas ao nível do *software*. Do lado do Zephyr, a análise dos ficheiros `pmp.c`, `switch.S` e `fatal.c` confirmou que a configuração PMP estava correta e que o registo `mcause` é escrito pelo *hardware* antes de qualquer código do *trap handler* executar. Do lado do compilador, a desassemblagem dos binários com recurso a `objdump` e `addr2line` confirmou que o `mepc` apontava corretamente para a instrução `sw` causadora do *fault*, sem qualquer instrução de *load* gerada previamente que pudesse justificar o `mcause` observado.

Excluído o *software*, a análise passou para o *RTL*, em particular para o módulo `pmp_data_if.sv`, onde o `mcause` e o `mtval` são gerados para acessos de dados. Este módulo decide a causa da exceção com base em dois sinais principais: `lsu_is_store_i`, que distingue entre um *Store Access Fault* e um *Load Access Fault*, e `lsu_vaddr_i`, que é copiado para o `mtval` em caso de violação. A Tabela 5.1 descreve os sinais relevantes para este diagnóstico.

Tabela 5.1: Sinais relevantes no diagnóstico da dessincronização PMP

Sinal	Descrição
<code>st_translation_req</code>	Combinacional. Pedido de <i>store</i> ;
<code>mmu_vaddr</code>	Combinacional. Endereço do acesso atual; com MMU sem tradução, igual ao físico.
<code>pmp_translation_valid</code>	A 1 no ciclo em que o PMP avalia o acesso.
<code>pmp_is_store</code>	Registado. <code>st_translation_req</code> atrasado 1 ciclo; estável na avaliação.
<code>pmp_vaddr</code>	Registado. <code>mmu_vaddr</code> atrasado 1 ciclo; estável na avaliação.
<code>lsu_is_store_i</code>	Entrada do <code>pmp_data_if</code> : 1 = escrita, 0 = leitura.
<code>lsu_vaddr_i</code>	Entrada do <code>pmp_data_if</code> : endereço copiado para <code>mtval</code> .

O problema detetado situa-se na dessincronização temporal (*timing*) com que estes sinais chegam ao módulo de verificação PMP. Na configuração utilizada, devido à presença da MMU, o acesso passa sempre pelo bloco `gen_mmu` do `load_store_unit.sv`, mesmo sem tradução ativa (`satp` a zero), o que introduz um ciclo de atraso no fluxo de dados.

O erro de omissão no *design* do processador reside na assimetria de registo destes sinais. Três dos sinais que alimentam o `pmp_data_if` (`lsu_valid_i`, `lsu_paddr_i` e `lsu_exception_i`) são *outputs* diretos da MMU e chegam registados, com o atraso de um ciclo. Contudo, os sinais `lsu_is_store_i` e `lsu_vaddr_i` não são expostos pela MMU. O código do `load_store_unit.sv` ligava-os diretamente aos sinais combinacionais `st_translation_req` e `mmu_vaddr`.

Como consequência direta desta ligação combinacional, quando a validação PMP ocorre, já com o atraso de um ciclo introduzido pela MMU, os sinais combinacionais já alteraram o seu estado na *pipeline*. O sinal `st_translation_req`, gerado num `always_comb` na *Finite State Machine* (FSM) da *store unit*, já caiu a zero devido à transição de `IDLE` para `VALID_STORE`, e o `mmu_vaddr` já reflete a instrução seguinte. Isto explica o preenchimento incorreto de `mcause` e `mtval`. O `mepc` não é afetado por esta assimetria porque provém do *commit stage*, sendo gerado a partir do PC guardado no *scoreboard* no momento da emissão da instrução. A Figura 5.1 evidencia esta dessincronização em simulação, com os sinais combinacionais já alterados no ciclo em que ocorre a validação.

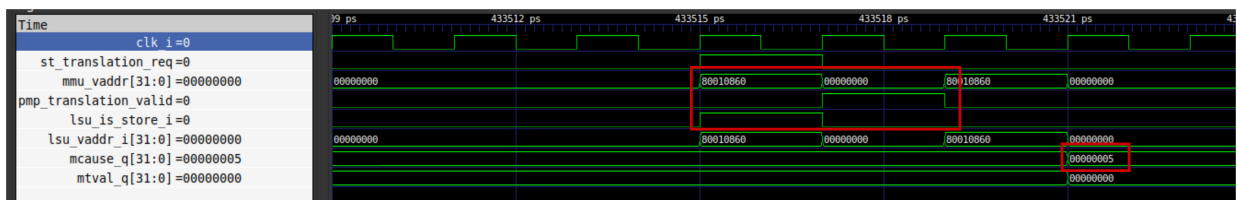


Figura 5.1: Formas de onda antes da correção

Esta dessincronização introduz uma vulnerabilidade crítica de segurança: com o sinal `lsu_is_store_i` a zero no momento da validação, o PMP avalia incorretamente o acesso como uma leitura (`ACCESS_READ`). Uma instrução de *store* direcionada a uma região com permissões exclusivas de leitura e execução (`R|X`), como a secção `.text` do *kernel*, é assim erroneamente autorizada pelo PMP e executada. Este comportamento foi confirmado através de um teste dirigido, no qual um `sw` para a área de código do *kernel* foi concluído com sucesso, corrompendo o código privilegiado de forma silenciosa enquanto a

thread continuava a executar normalmente.

5.2.2 Correção

A correção foi aplicada apenas no `load_store_unit.sv`, sem alterações ao `cva6_mmu.sv`. Uma vez que os dois sinais já eram registados corretamente dentro da MMU mas não expostos, optou-se por replicar esse registo no exterior. No bloco `gen_mmu`, a seguir à instância da MMU, foi adicionado o registo da Listagem 5.1, que atrasa `st_translation_req` e `mmu_vaddr` um ciclo, em sincronia com os restantes sinais. No bloco `gen_no_mmu`, onde já existia um registo equivalente para os outros três sinais, os dois sinais em falta foram adicionados ao mesmo bloco.

```

1 always_ff @(posedge clk_i or negedge rst_ni) begin
2   if (!rst_ni) begin
3     pmp_is_store <= 1'b0;
4     pmp_vaddr    <= '0;
5   end else begin
6     pmp_is_store <= st_translation_req;
7     pmp_vaddr    <= mmu_vaddr;
8   end
9 end

```

Listing 5.1: Registo adicionado no bloco `gen_mmu` do `load_store_unit.sv`.

O wiring do `pmp_data_if` foi então corrigido para utilizar os novos sinais registados em vez dos combinacionais (Listagem 5.2).

```

1 // antes
2 .lsu_is_store_i (st_translation_req),
3 .lsu_vaddr_i    (mmu_vaddr),
4
5 // depois
6 .lsu_is_store_i (pmp_is_store),
7 .lsu_vaddr_i    (pmp_vaddr),

```

Listing 5.2: Correção do wiring do `pmp_data_if`: sinais combinacionais substituídos pelos registados.

Após a correção, todos os cenários de violação por escrita passaram a gerar `mcause = 0x7` com `mtval` a apontar para o endereço correto do acesso ilegal, e os `stores` para regiões R/X passaram a ser bloqueados. A Figura 5.2 mostra o resultado em simulação, com os sinais registados sincronizados e o reporte da exceção já conforme a especificação. O impacto desta correção nos resultados é discutido no Capítulo 6. Corrigido o reporte de violações, a secção seguinte descreve os cenários de demonstração construídos sobre esta base.

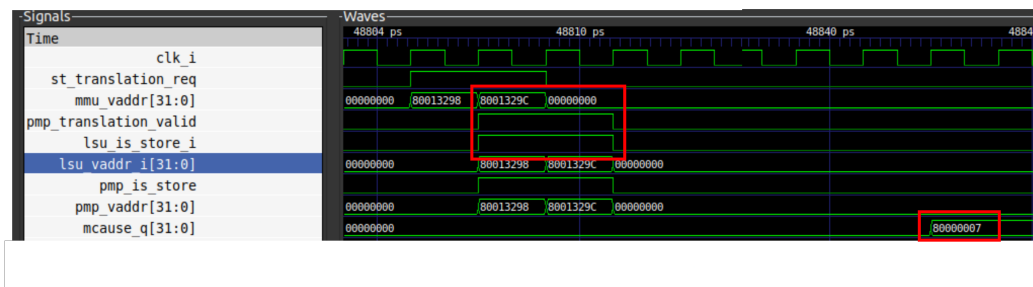


Figura 5.2: Formas de onda após a correção

5.3 Implementação dos Cenários de Demonstração

Os cenários de demonstração concretizam o modelo de ameaça definido na Secção 3.3. Cada cenário é implementado de forma equivalente nos dois RTOSes, sobre o mesmo hardware, diferindo

apenas na presença ou ausência de isolamento PMP, o que permite expor lado a lado o efeito da sua configuração face à sua ausência.

Foram definidos quatro cenários. Os três primeiros correspondem a acessos adversariais intencionais, dirigidos a alvos distintos: a *stack* privada de outra *task*, uma variável global no espaço do kernel, e a estrutura de controlo de uma *task* (TCB). O quarto corresponde a um acesso não intencional, resultante de um erro de programação que provoca escrita fora dos limites de um *array* local. Todos assentam em acessos de escrita, por serem diretamente observáveis através da corrupção de dados. Acessos de leitura ilegais são bloqueados pelo mesmo mecanismo. A Tabela 5.2 resume o comportamento esperado em cada RTOS.

Tabela 5.2: Cenários de demonstração e comportamento esperado por RTOS.

Cenário	FreeRTOS (sem PMP)	Zephyr (com PMP)
Escrita em <i>stack</i> alheia	Escrita bem-sucedida; password corrompida sem deteção	<i>Store Access Fault</i> ; sistema parado
Escrita em global do kernel	Escrita bem-sucedida; dados corrompidos sem deteção	<i>Store Access Fault</i> ; sistema parado
Corrupção do TCB	<i>Scheduler</i> opera com <i>stack pointer</i> inválido; sistema falha	<i>Store Access Fault</i> ; sistema parado
Bug acidental	<i>Overflow</i> silencioso; dados de outra <i>task</i> corrompidos	<i>Store Access Fault</i> ; sistema parado

5.3.1 Configuração de Isolamento no Zephyr

No Zephyr, o isolamento assenta no suporte nativo a *user mode*, ativado por `CONFIG_USERSPACE`. As *tasks* são criadas com a flag `K_USER` e executam em U-Mode, cada uma com uma entrada PMP dedicada à sua *stack*. Qualquer acesso fora dessa região, seja à *stack* de outra *task*, seja a memória do kernel, é intercetado pelo PMP e termina a execução do sistema, conforme o mecanismo descrito na Secção 2.1.2.

Os acessos adversariais que dependem de coordenação entre *tasks*, como a partilha de um endereço alvo, exigem uma região de memória acessível a ambas. Para esse efeito recorre-se ao mecanismo de partições do Zephyr. Uma partição partilhada (Listagem 5.3) é atribuída aos domínios das duas *tasks*, mantendo-se privadas as restantes regiões.

```

1 K_APPMEM_PARTITION_DEFINE(shared_part);
2
3 K_APP_DMEM(shared_part) volatile uint32_t leaked_addr = 0;
4 K_APP_DMEM(shared_part) volatile uint32_t shared_ready = 0;
```

Listing 5.3: Definição da partição partilhada e das variáveis de coordenação no Zephyr.

As *tasks* são criadas suspensas, o domínio é configurado, e apenas depois são iniciadas. Esta ordem garante que as entradas PMP estão configuradas antes da primeira instrução executada em U-Mode.

5.3.2 Escrita na Stack de Outra Task

Uma variável local não está protegida pelo simples facto de residir na *stack* de uma *task*. A Task A guarda uma password na sua *stack* e expõe o respetivo endereço numa variável partilhada, simulando um registo de depuração. A *task* adversarial lê esse endereço e tenta escrever diretamente na *stack* da Task A (Listagem 5.4).

```

1 volatile int *ptr = (volatile int *)leaked_addr;
2 *ptr = 0xDEAD;
```

Listing 5.4: Escrita na *stack* da Task A a partir do endereço exposto.

No FreeRTOS, com ambas as *tasks* em M-Mode, a escrita sucede e a password é corrompida sem detecção. No Zephyr, a *stack* da Task A tem uma entrada PMP exclusiva e o acesso da *task* adversarial gera um *Store Access Fault*.

5.3.3 Escrita em Variável Global do Kernel

A segunda demonstração visa uma variável global, que reside no espaço de memória do kernel e é referenciada por símbolo, sem necessidade de endereço exposto. A *task* adversarial escreve diretamente nessa variável.

```
1 /* Task Adv escreve diretamente na variavel global da Task A */
2 dados_task_a = 0xDEAD;
```

Listing 5.5: Escrita direta na variável global da Task A a partir da Task Adv.

No FreeRTOS, a variável é acessível a qualquer *task* sem restrição e a escrita sucede, passando a Task A a observar dados corrompidos. No Zephyr, a variável não pertence a nenhuma partição de *user* e permanece no espaço do kernel, fora do alcance da *task* em U-Mode; o acesso gera um *Store Access Fault*.

5.3.4 Corrupção do TCB

A terceira demonstração ataca a estrutura de controlo da *task*, o alvo com maior potencial de dano por conter o estado de que o *scheduler* depende. A *task* adversarial obtém a referência para a estrutura da Task A e escreve no seu primeiro campo, o *stack pointer*.

```
1 /* Task Adv obtem o handle da Task A e escreve no primeiro campo do TCB */
2 uint32_t *tcb = (uint32_t *)handleTaskA;
3 tcb[0] = 0xDEADBEEF; /* primeiro campo = stack pointer */
```

Listing 5.6: Corrupção do TCB da Task A: cast do handle para ponteiro e escrita no primeiro campo.

No FreeRTOS, a escrita sucede e o *scheduler* passa a operar com um *stack pointer* inválido, levando o sistema a um estado indefinido. No Zephyr, a estrutura de controlo reside no espaço do kernel e o acesso gera um *Store Access Fault* antes de qualquer corrupção.

5.3.5 Bug Acidental

A quarta demonstração não envolve intenção maliciosa. Uma *task* declara um array local de oito posições mas, por erro de constante, percorre 512 iterações, escrevendo para além dos limites do array (Listagem 5.7). Trata-se de um erro frequente em C embebido.

```
1 volatile char config[8];
2 for (volatile int i = 0; i < 512; i++) { /* deveria ser 8 */
3     config[i] = 0;
4 }
```

Listing 5.7: Escrita fora dos limites por erro de constante no índice do ciclo.

No FreeRTOS, o *overflow* ultrapassa silenciosamente os limites do array e atinge dados de outra *task*, sem detecção. No Zephyr, a escrita é contida assim que excede a região PMP atribuída à *stack*, gerando um *Store Access Fault*. O isolamento PMP é, por isso, independente da intenção: protege a integridade do sistema tanto contra acessos maliciosos como contra erros de programação, classe de falhas particularmente frequente nos domínios *safety-critical* referidos na Secção 1.1.

5.4 Extensão do FreeRTOS para U-Mode

O port original do FreeRTOS para RISC-V executa todas as *tasks* em M-Mode. Para as lançar em U-Mode, foram necessárias três alterações.

A primeira foi definir, na inicialização da *stack* de cada *task*, o valor de MPP no *mstatus* como 00 (U-Mode). Uma vez que o *context switch* termina sempre com *mret*, e o *mret* lê o MPP do *mstatus* restaurado da *stack*, esta alteração garante que todas as *tasks* arrancam e retomam em U-Mode sem qualquer modificação adicional ao mecanismo de *context switch* existente.

A segunda foi configurar, antes do primeiro *mret*, uma entrada PMP que cobre a totalidade do espaço de memória com permissões de leitura, escrita e execução (Listagem 5.8). Esta configuração é necessária porque, assim que o processador entra em U-Mode, o PMP passa a verificar todos os acessos. Sem nenhuma entrada válida, a primeira instrução da *task* geraria imediatamente um *access fault*. Uma vez que o objetivo do FreeRTOS neste projeto é servir de base sem isolamento, a entrada PMP mantém-se inalterada ao longo de toda a execução.

```

1  /* PMP: permite acesso total em U-mode */
2  li    t0, 0x7FFFFFFF
3  csrw  pmpaddr0, t0
4  li    t0, 0x1F
5  csrw  pmpcfg0, t0

```

Listing 5.8: Configuração da entrada PMP com acesso total antes do primeiro *mret*.

A terceira alteração foi ao handler de exceções síncronas. Quando uma *task* em U-Mode invoca *taskYIELD*, é executada internamente uma instrução *ecall*, que gera *mcause* = 8 (*ecall* de U-Mode). O handler original, apresentado na Listagem 5.9, apenas tratava *mcause* = 11 (*ecall* de M-Mode), encaminhando-o para o *context switch*; qualquer outro valor era encaminhado para o *freertos_risc_v_application_exception_handler*, que fica preso num ciclo sem saída. Foi adicionado o tratamento de *mcause* = 8 com o mesmo encaminhamento, conforme a Listagem 5.10.

```

1  handle_exception:
2      /* a0 contains mcause. */
3      li    t0, 11                      /* 11 == environment call. */
4      bne a0, t0, application_exception_handler
5      call vTaskSwitchContext
6      j processed_source
7
8  application_exception_handler:
9      call freertos_risc_v_application_exception_handler
10     j processed_source                /* No other exceptions handled yet. */
11
12 processed_source:
13     portcontextRESTORE_CONTEXT

```

Listing 5.9: Handler de exceções síncronas original: apenas trata *ecall* de M-Mode (*mcause* = 11).

```

1  handle_exception:
2      /* a0 contains mcause. */
3      li    t0, 11
4      beq   a0, t0, do_switch      /* ecall de M-mode */
5      li    t0, 8
6      beq   a0, t0, do_switch      /* ecall de U-mode */
7      j     application_exception_handler
8
9  do_switch:
10     call  vTaskSwitchContext
11     j     processed_source
12
13 application_exception_handler:
14     call  freertos_risc_v_application_exception_handler
15     j     processed_source        /* No other exceptions handled yet. */
16
17 processed_source:
18     portcontextRESTORE_CONTEXT

```

Listing 5.10: Handler de exceções síncronas modificado: trata ecall de M-Mode (`mcause = 11`) e de U-Mode (`mcause = 8`).

Esta extensão evidencia uma limitação fundamental do FreeRTOS: o *kernel* foi concebido para executar em M-Mode, e as suas funções internas acedem diretamente a CSRs, registos privilegiados inacessíveis em U-Mode. Qualquer chamada a essas funções a partir de uma *task* em U-Mode gera `mcause = 2` (*Illegal Instruction*), terminando o sistema.

Na prática, isto limitou as primitivas disponíveis para as *tasks* de demonstração. Funções como `vTaskDelay`, que internamente manipulam CSRs do timer, não podem ser invocadas em U-Mode; o `taskYIELD` é a exceção porque se limita a executar `ecall`, uma instrução não privilegiada. Por este motivo, o suporte a timers foi desativado e os módulos de demonstração que dependem de primitivas privilegiadas foram removidos do projeto.

Esta limitação é precisamente o que o Zephyr resolve com o seu suporte nativo a *user mode*: todas as operações privilegiadas são mediadas por chamadas ao sistema, tornando a fronteira M-Mode/U-Mode funcional e sustentável, conforme descrito na Secção 4.3.

6. Testes e Discussão

6.1 Metodologia de Testes

A validação foi conduzida em dois ambientes complementares: simulação *cycle-accurate* do CVA6 via Verilator e execução em hardware real na FPGA Genesys2 (Secção 5.1). Cada cenário, definido no Capítulo 5, foi executado sobre o FreeRTOS e o Zephyr, com o mesmo hardware e o mesmo tipo de *tasks*, diferindo apenas na presença ou ausência de configuração PMP. O critério de sucesso é, no FreeRTOS, a corrupção efetiva dos dados sem detecção; no Zephyr, um *Store Access Fault* (*mcause* = 0x7) com paragem do sistema antes de qualquer corrupção.

As demonstrações físicas com o motor (Secções 6.5 e 6.6) foram registadas em vídeo, disponíveis em: [demonstração em vídeo](#).

6.2 Simulação: Escrita em Variável Global do Kernel

Cenário da Secção 5.3.3, validado em simulação. No FreeRTOS, a escrita na variável global sucede sem detecção e a *Task A* passa a observar o valor corrompido. No Zephyr, o PMP interceta o acesso e gera um *Store Access Fault* (Figura 6.1).

```
*** [rvf_tracer] INFO: Loading binary :
*** [rvf_tracer] INFO: tohost_addr: 00000000
*** [rvf_tracer] WARNING: No valid address of 'tohost'

warning: tohost and fromhost symbols not in ELF; can't
init_uart
[MAIN] hardware ok
[MAIN] mtvec configurado
[MAIN] a entrar em main_blinky
[TASK A] dados = 42 (OK)
[TASK A] dados = 42 (OK)
[TASK A] dados = 42 (OK)
[ADV] a corromper dados da Task A...
[TASK A] dados CORROMPIDOS!
[TASK A] dados CORROMPIDOS!
[TASK A] dados CORROMPIDOS!
[ADV] a corromper dados da Task A...
[TASK A] dados CORROMPIDOS!
[TASK A] dados CORROMPIDOS!
[TASK A] dados CORROMPIDOS!
[ADV] a corromper dados da Task A...
[TASK A] dados CORROMPIDOS!
[TASK A] dados CORROMPIDOS!
[TASK A] dados CORROMPIDOS!
[ADV] a corromper dados da Task A...
[TASK A] dados CORROMPIDOS!
```

(a) FreeRTOS: acesso não detetado.

```
Listening on port 35833
*** [rvf_tracer] INFO: Loading binary :
*** [rvf_tracer] INFO: tohost_addr: 0000000000000000
*** [rvf_tracer] WARNING: No valid address of 'tohost'

warning: tohost and fromhost symbols not in ELF; can't
*** Booting Zephyr OS build v4.4.0-538-g7bbe9da0892c **
[TASK A] a correr
[TASK A] a correr
[TASK A] a correr
[ADV] a tentar corromper dados_task_a

mcause: 7, Store/AMO access fault
mtval: 80010860
a0: 00000000 t0: 00000047
a1: 00000007 t1: 80012770
a2: 800127a4 t2: 00000000
a3: 00000001 t3: 00000003
a4: 80011000 t4: 00000000
a5: 0000dead t5: 00000000
a6: 00000003 t6: 00000000
a7: 20004207
sp: 800127c0
ra: 8000274c
mepc: 80000c50
mstatus: 00000080

s0: 00000000 s6: 00000000
s1: 80000c14 s7: 00000000
s2: 00000000 s8: 00000000
s3: 80000c14 s9: 00000000
s4: 00000000 s10: 00000000
s5: 00000000 s11: 00000000

call trace:
```

(b) Zephyr: *Store Access Fault*.

Figura 6.1: Simulação Verilator: escrita em variável global do kernel.

6.3 Hardware: Bug Acidental

Cenário da Secção 5.3.5, validado em hardware. No FreeRTOS, o *overflow* ultrapassa silenciosamente os limites do array e corrompe dados de outra *task* (Figura 6.2). No Zephyr, o PMP interceta a escrita ao ultrapassar a região da *stack* e gera um *Store Access Fault*.

O *mepc* aponta o endereço da instrução em falta (0x80000c74); o *objdump* confirma tratar-se de um *sb* (*store*) e o *addr2line* mapeia-o para *main_z_bug_acidental.c*:54 (Figura 6.3), que corresponde a *config[i] = 0*, a escrita fora dos limites.

```
[TASK A] valores iniciais: 10, 20, 30, 40
[TASK A] dados OK
[TASK A] dados OK
[TASK A] dados OK
[TASK A] dados OK
[TASK A] dados OK
[BUG] a escrever config (bug: loop de 8, devia ser 4)...
[BUG] isto vai sobrescrever dados da Task A!
[TASK A] dados OK
[TASK A] dados OK
[TASK A] dados OK
[TASK A] dados OK
[TASK A] dados CORROMPIDOS!
[TASK A] novos valores: [0]=0x0000BEEF [1]=0x00000014 [2]=0x0000001E [3]=0x0000002A
```

Figura 6.2: FreeRTOS: *overflow* silencioso; dados de outra *task* corrompidos sem detecção.

```

Hit any key to enter update mode ..... booting!
[init SPI
status: 0x0000000000000025
status: 0x0000000000000025
SPI initialized!
initializing SD...
*** Booting Zephyr OS build v4.0-0.538-g7bbe9da0892c ***
[TASK_A] a correr normalmente
[TASK_A] a correr normalmente
[TASK_A] a correr normalmente
[BUG] a escrever config (bug: 512 Itaceracos, devia ser 8)...

ncause: 7, Store/AMO access fault
ntval: 80012800
a0: 00000000          t0: 00000047
a1: 00000001          t1: 80011268
a2: 80012794          t2: 00000000
a3: 00000001          t3: 00000003
a4: 000001ff          t4: ffffffff
a5: 80012808          t5: ffffffff
a6: 00000003          t6: ffffffff
a7: 20004207
sp: 800127b0
mcp: 80000c48
mepc: 80000c74
mstatus: 00000000

s0: 00000000          s6: 00000000
s1: 80000c14          s7: 00000000
s2: 00000000          s8: 00000000
s3: 80000c14          s9: 00000000
s4: 00000000          s10: 00000000
s5: 00000000          s11: 00000000

call trace:

```

```
mendes@mendes-Katana-GF66-12UG5-5: /home/mendes$ cat /home/mendes/.bashrc
zephyr-elf objdump -d /home/mendes/Pl3/zephyrproject/build/zephyr/zephyr_elf | grep -B10 -A10 "80000c74:"
80000c4c: 00f75663      bge     a4,a5,80000c68 <task_buggy_entry+0x54>
80000c50: 01c12083      lw      ra,78(sp)
80000c54: 8000a537      lui     a0,0x80000000
80000c58: 78850513      addui   a0,a0,1928 # 8000a788 <__clz_tab+0x140>
80000c5c: 02610113      addi    sp,sp,32
80000c60: 00001317      auipc   t1,0x1
80000c64: 75210067      jr      t1,1876(t1) # 800023b4 <printrc>
80000c68: 00412783      lw      a5,a4(sp)
80000c6c: 01078793      addui   a5,a5,16 # 10010 <_rom_region_size+0x10>
80000c70: 002787b3      addi    a5,a5,sp
80000c74: 002787b3      sb      zero,a5(sp)
80000c78: 00412783      lw      a5,a4(sp)
80000c7c: 00178793      addi    a5,a5,1
80000c80: 00f12223      sw      a5,a4(sp)
80000c84: fc1ff06f      j       80000c48 <task_buggy_entry+0x34>

80000c88 <task_a_entry>:
80000c8c: ffe01113      addi    sp,sp,-16
80000c90: sw      s0,8(sp)
80000c94: sw      ra,12(sp)
80000c98: 00000000      lui     s0,0x80000000
mendes@mendes-Katana-GF66-12UG5-5: /home/mendes$ cat /home/mendes/.bashrc
zephyr-elf add2line -e /home/mendes/Pl3/zephyrproject/build/zephyr/zephyr_elf -f 80000c74
task_buggy_entry at /home/mendes/Pl3/zephyr_apps/demos_seguraca/src/main_2_bug_accidental.c:54
mendes@mendes-Katana-GF66-12UG5-5: /
```

(a) *Dump* de registros

```
(b) objdump e addr2line
```

Figura 6.3: Zephyr (hardware): detecção pelo PMP e diagnóstico simbólico a partir do mepc.

6.4 Hardware: Extensão do FreeRTOS para U-Mode

Validação da extensão da Secção 5.4. Na configuração base, com *tasks* em M-Mode, uma *task* lê o `mstatus` sem restrição (Figura 6.4a). Após a extensão, com *tasks* em U-Mode, uma chamada a função do kernel que acede a CSRs gera uma exceção encaminhada para o `application_exception_handler`, onde o fluxo fica preso no ciclo `j`. (Figura 6.4b).

```
[info] Listening on port 4444 for telnet connections...
[warn:] Prefer gdb command "target extended-remote :3333" instead of "target remote :3333"
```

```
# mmdes@mendes-katana-GF60-IUGCS-VPI93I87C0LHJN@bin tok3as
~/Home/mmdes/Pt11/freertos/FreetDS/sources/tasks.c
   6111 |             static void prvCheckTaskWaitingTermination( void ) {
   6112 |                 /* THIS FUNCTION IS CALLED FROM THE RTOS IDLE TASK */
   6113 |                 if ( INCLUDE_vTaskDelete == 1 )
   6114 |                     TCB_t * pTCB;
   6115 |                 /* uxDeletedTasksWaitingCleanUp is used to prevent taskENTER_CRITICAL()
   6116 |                  being called too often in the idle task */
   6117 |                 while (uxDeletedTasksWaitingCleanUp != (UBaseType_t) 0U) {
   6118 |                     #if ( configNUMBER_OF_CORES == 1 )
   6119 |                         taskENTER_CRITICAL();
   6120 |                     } else {
   6121 |                         /* MISRA Ref 11-5:3 [void pointer assignment] */
   6122 |                         /* More details at: https://github.com/FreetDS/FreetDS-Kernel/
   6123 |                          * /commit/nitros_c_2012_10c_5_validation */
   6124 |                         pTCB = listGET_OWNER_OF_HEAD_ENTRY( &xPendingTaskWaitingTermination );
   6125 |                         (void) uxListRemove(&pTCB->xStaticListItem);
   6126 |                     }
   6127 |                 }
   6128 |             }
   6129 |         }
   6130 |     }
```

```
Remote Remote target in: prvCheckTaskWaitingTermination           L0179 PC: 0x000020be
(gdb) bu! Layout Src
(gdb) bu! Focus end
Focus set to end window.
(gdb) c
```

(a) M-Mode: *task* lê *mstatus* sem restrição.

```

Input: [
[16:08:56:979] <break>
[16:09:50:576] Hello World! %s
[16:09:50:576] Hit any key to enter update mode ..... booting! %s
[16:09:55:593] init SPI... %s
[16:09:55:593] status: 0x0000000000000025 %s
[16:09:55:593] status: 0x0000000000000025 %s
[16:09:55:593] SPI initialized! %s
[16:09:55:593] initializing SD... %s
[16:15:42:027] init_uart %s
[16:15:42:027] [TASK CSR] a tentar ler mstatus... %s
]

```

(b) U-Mode: exceção presa no *handler* (j. 1.).

Figura 6.4: Extensão do FreeRTOS para U-Mode: M-Mode vs bloqueio em U-Mode ao executar instruções privilegiadas.

6.5 Demonstração Física: Escrita na *Stack* de Outra *Task*

As duas demonstrações seguintes foram concretizadas em hardware com um motor DC real, tornando visível o impacto de uma violação de memória sobre um atuador. A FPGA Genesys2 gera um sinal *Pulse Width Modulation* (PWM) num pino *General Purpose Input/Output* (GPIO) que alimenta um *driver* de ponte H L298N, o qual fornece potência ao motor a partir de uma fonte externa; o *duty cycle*, definido em software, determina a velocidade de rotação (Figura 6.5).

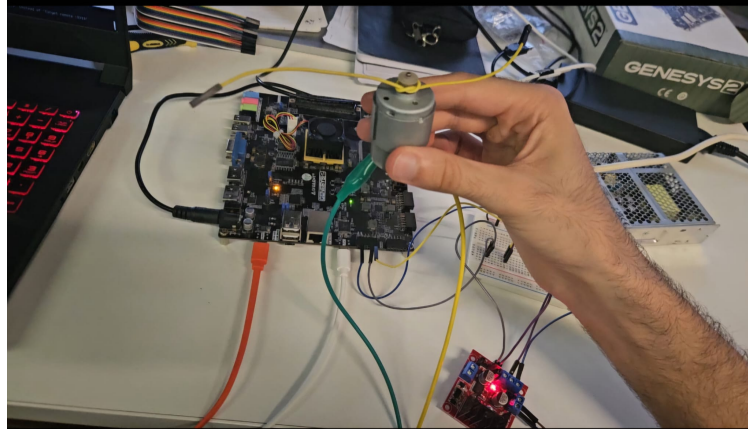


Figura 6.5: Montagem física: FPGA Genesys2 a gerar PWM via GPIO, ponte H L298N e motor DC.

Neste primeiro cenário (Secção 5.3.2), a *task* legítima mantém o `pwm_duty` do motor na sua *stack* e a *Task* Adv sobrescreve-o. No FreeRTOS, a escrita sucede sem deteção: o `pwm_duty` é corrompido e o motor abrandando (Figura 6.6a). No Zephyr, o PMP interceta a escrita antes de a corrupção ocorrer, gerando um *Store Access Fault*; o sistema para de forma controlada em `arch_system_halt` (Figura 6.6b), impedindo a alteração do `pwm_duty` por uma *task* não autorizada. Como o *halt* suspende toda a execução, a geração do sinal PWM cessa e o motor fica num estado fixo, determinado pelo nível do pino no instante da paragem. A proteção do PMP atua, assim, ao nível da integridade da memória: garante que o valor legítimo não é corrompido, ainda que a resposta à violação (a paragem do sistema) deva ser complementada, num sistema de produção, por um tratamento de erro que coloque o atuador num estado seguro.

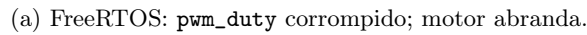
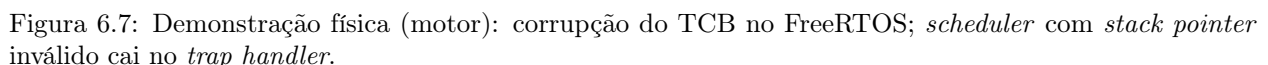


Figura 6.6: Demonstração física (motor): escrita na *stack* de outra *task*.

Cenário da Secção 5.3.4, concretizado na mesma demonstração física. A *Task Adv* escreve no primeiro campo do TCB (o *stack pointer*) da *task* do motor.

No FreeRTOS, a escrita sucede: o *scheduler* passa a operar com um *stack pointer* inválido e o sistema cai no `freertos_risc_v_trap_handler`, perdendo o controlo do motor (Figura 6.7). No Zephyr, o TCB reside no espaço do kernel e o acesso gera um *Store Access Fault* antes de qualquer corrupção.



6.7 Validação dos Objetivos

A Tabela 6.1 mapeia cada objetivo (Secção 1.2) à evidência que o valida.

Tabela 6.1: Validação dos objetivos do trabalho.

Obj.	Descrição (resumida)	Evidência
1	Compreender níveis de privilégio e PMP no CVA6	Cap. 2, Sec. 5.2
2	Configurar o FreeRTOS para o processador CVA6	Sec. 6.4
3	Estender o FreeRTOS para U-Mode sem PMP	Sec. 6.4
4	Configurar o Zephyr com PMP nativo por <i>task</i>	Sec. 6.2, 6.3, 6.5–6.6
5	<i>Tasks</i> adversariais e efeito do isolamento	Sec. 6.5–6.6
6	Comparar FreeRTOS vs Zephyr (Simulação + FPGA)	Todo o Cap. 6

Os resultados confirmam que a configuração correta do PMP garante isolamento efetivo entre *tasks*, com detecção imediata de violações em simulação e em hardware, ao passo que a sua ausência permite a corrupção sem detecção. O isolamento é independente da intenção, protegendo tanto contra acessos maliciosos como contra erros de programação, conforme tornado tangível pela demonstração física com o motor.

7. Conclusão

Este trabalho avaliou e demonstrou a importância do isolamento de memória em microcontroladores, assim como os riscos associados à sua ausência em sistemas embebidos de tempo real. Para o efeito, foram implementadas *tasks* com diversos comportamentos adversariais, integradas e comparadas em dois sistemas operativos de tempo real (RTOS) distintos: o FreeRTOS e o Zephyr. A avaliação recorreu ao mecanismo de *Physical Memory Protection* (PMP) da arquitetura RISC-V, com validação sobre o processador CVA6.

No FreeRTOS, o suporte original foi estendido para executar *tasks* em U-Mode, o que evidenciou que a separação de níveis de privilégio, por si só, é insuficiente para garantir o isolamento: sem entradas PMP restritivas, o espaço de endereçamento permanece integralmente exposto. Nos cenários avaliados na configuração base em M-Mode, verificou-se a corrupção de dados críticos e do próprio sistema sem qualquer ação ou deteção por parte do *hardware*. No Zephyr, o suporte nativo ao PMP assegurou isolamento efetivo em todos os cenários adversariais: cada violação foi intercetada antes de causar dano, resultando numa paragem controlada com registo da causa e do endereço acedido ilegalmente.

Durante a validação, foi identificada e corrigida uma anomalia no *RTL* do CVA6, que classificava escritas ilegais como leituras e permitia a sua execução sem exceção em determinadas regiões de memória. A correção, aplicada na Load Store Unit, restabeleceu o comportamento previsto pela especificação do PMP, constituindo uma contribuição direta para a robustez deste núcleo de grau industrial.

A demonstração física com o controlo de um motor DC numa FPGA Genesys2 tornou o impacto destas vulnerabilidades tangível: sob o FreeRTOS, a corrupção da *stack* da *task* do motor alterou o seu comportamento de forma silenciosa; sob o Zephyr, a mesma tentativa foi bloqueada, forçando a paragem segura do sistema antes de qualquer dano.

Os resultados confirmam que o PMP, quando corretamente configurado e gerido pelo sistema operativo através de reconfiguração dinâmica, garante o isolamento efetivo entre os componentes da pilha de *software*. Esta proteção atua de forma independente da intenção do código, salvaguardando o sistema tanto contra acessos maliciosos como contra erros de programação, comuns em C embebido.

7.1 Trabalho Futuro

Como continuação do trabalho desenvolvido, destacam-se três direções principais de trabalho futuro:

1. **Implementação de isolamento PMP por *task* no FreeRTOS:** Reconfigurar as entradas PMP a cada *context switch* de forma análoga ao comportamento nativo do Zephyr, permitindo ao FreeRTOS oferecer isolamento real entre *tasks* e eliminando a exposição identificada na base;
2. **Suporte integral ao U-Mode no FreeRTOS:** Desenvolver um mecanismo de chamadas ao sistema (*system calls*) para mediar o acesso a funções internas do RTOS que manipulam registos privilegiados, permitindo que primitivas como `vTaskDelay` e os temporizadores operem normalmente a partir de *tasks* não privilegiadas;
3. **Mecanismos de tolerância a falhas no Zephyr:** Desenvolver uma resposta mais robusta a violações de memória onde, em vez de uma paragem total do sistema após um *access fault*, o sistema operativo termine apenas a *task* ofensora e retome o escalonamento normal, aumentando a resiliência do sistema.

Bibliografia

- [1] Profience. (2025) Software as the new root cause – the rise of infotainment and software-driven recalls. [Online]. Available: <https://profience.com/2025-05-13-software-as-the-new-root-cause-the-rise-of-infotainment-and-software-driven-recalls/>
- [2] U.S. Food and Drug Administration (FDA), “Medical device recalls: Software-related failures and safety alerts,” <https://www.fda.gov/medical-devices/medical-device-safety/medical-device-recalls>, 2024, acedido em: 13 de Junho de 2026.
- [3] RISC-V International, *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 20241017*, 2024. [Online]. Available: <https://docs.riscv.org/reference/isa/priv/priv-intro.html>
- [4] OpenHW Group, “CVA6 – a highly configurable 6-stage RISC-V core,” 2024. [Online]. Available: <https://github.com/openhwgroup/cva6>
- [5] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture, Version 20240411*, 2024. [Online]. Available: <https://docs.riscv.org/reference/isa/unpriv/unpriv-index.html>
- [6] Amazon Web Services, “FreeRTOS – real-time operating system for microcontrollers,” 2024. [Online]. Available: <https://www.freertos.org>
- [7] Free RTOS, “Using FreeRTOS on RISC-V microcontrollers,” 2024. [Online]. Available: <https://www.freertos.org/Using-FreeRTOS-on-RISC-V>
- [8] Zephyr Project, “Zephyr RTOS a scalable open source RTOS,” 2024. [Online]. Available: <https://github.com/zephyrproject-rtos/zephyr>
- [9] RISC-V, “RISC-V support in zephyr PMP and user mode,” 2024. [Online]. Available: <https://docs.zephyrproject.org/latest/hardware/arch/risc-v.html>
- [10] W. Zhou, L. Guan, P. Liu, and Y. Zhang, “Good motive but bad design: Why ARM MPU has become an outcast in embedded systems,” *arXiv preprint arXiv:1908.03638*, 2019. [Online]. Available: <https://arxiv.org/abs/1908.03638>
- [11] C. Larman, “Secure task management in FreeRTOS: A RISC-V core approach with physical memory protection,” Master’s Thesis, Delft University of Technology, 2024. [Online]. Available: <https://resolver.tudelft.nl/uuid:25ac1dc8-7d80-485b-be33-b4f43b6c379e>
- [12] K. Cheang, C. Rasmussen, D. Lee, D. W. Kohlbrenner, K. Asanović, and S. A. Seshia, “Verifying RISC-V physical memory protection,” *arXiv preprint arXiv:2211.02179*, 2022. [Online]. Available: <https://arxiv.org/abs/2211.02179>
- [13] B. Sá, L. Valente, J. Martins, D. Rossi, L. Benini, and S. Pinto, “CVA6 RISC-V virtualization: Architecture, microarchitecture, and design space exploration,” *arXiv preprint arXiv:2302.02969*, 2023. [Online]. Available: <https://arxiv.org/abs/2302.02969>
- [14] Wilson Snyder and contributors, “Verilator – fast, free Verilog/SystemVerilog simulator,” 2024. [Online]. Available: <https://www.veripool.org/verilator/>
- [15] Digilent Inc., “Genesys 2 AMD Kintex-7 FPGA development board,” 2024. [Online]. Available: <https://digilent.com/reference/programmable-logic/genesys-2/start>
- [16] The OpenOCD Project, “OpenOCD – open on-chip debugger,” 2024. [Online]. Available: <https://openocd.org>